

INTERRUPCIONES

Arquitectura de Computadores y Ensambladores 2

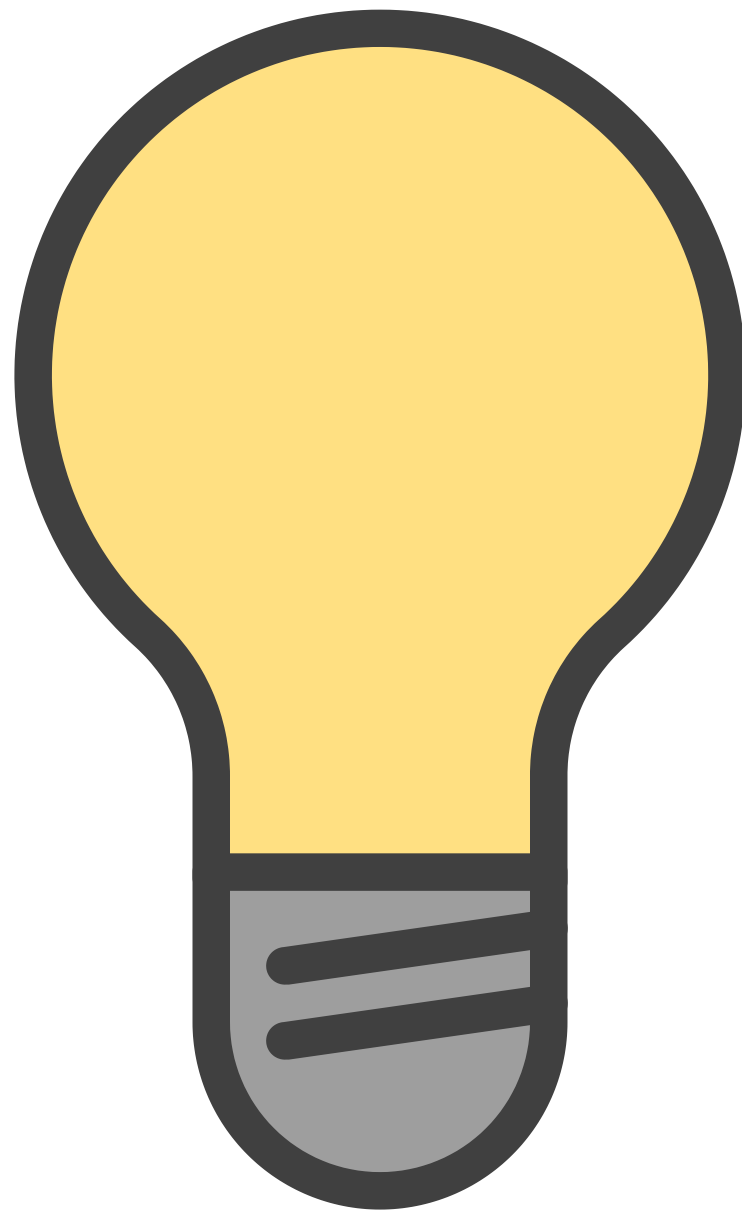
Escuela de Ingeniería de Ciencias Y Sistemas

Facultad de Ingeniería

Universidad de San Carlos de Guatemala



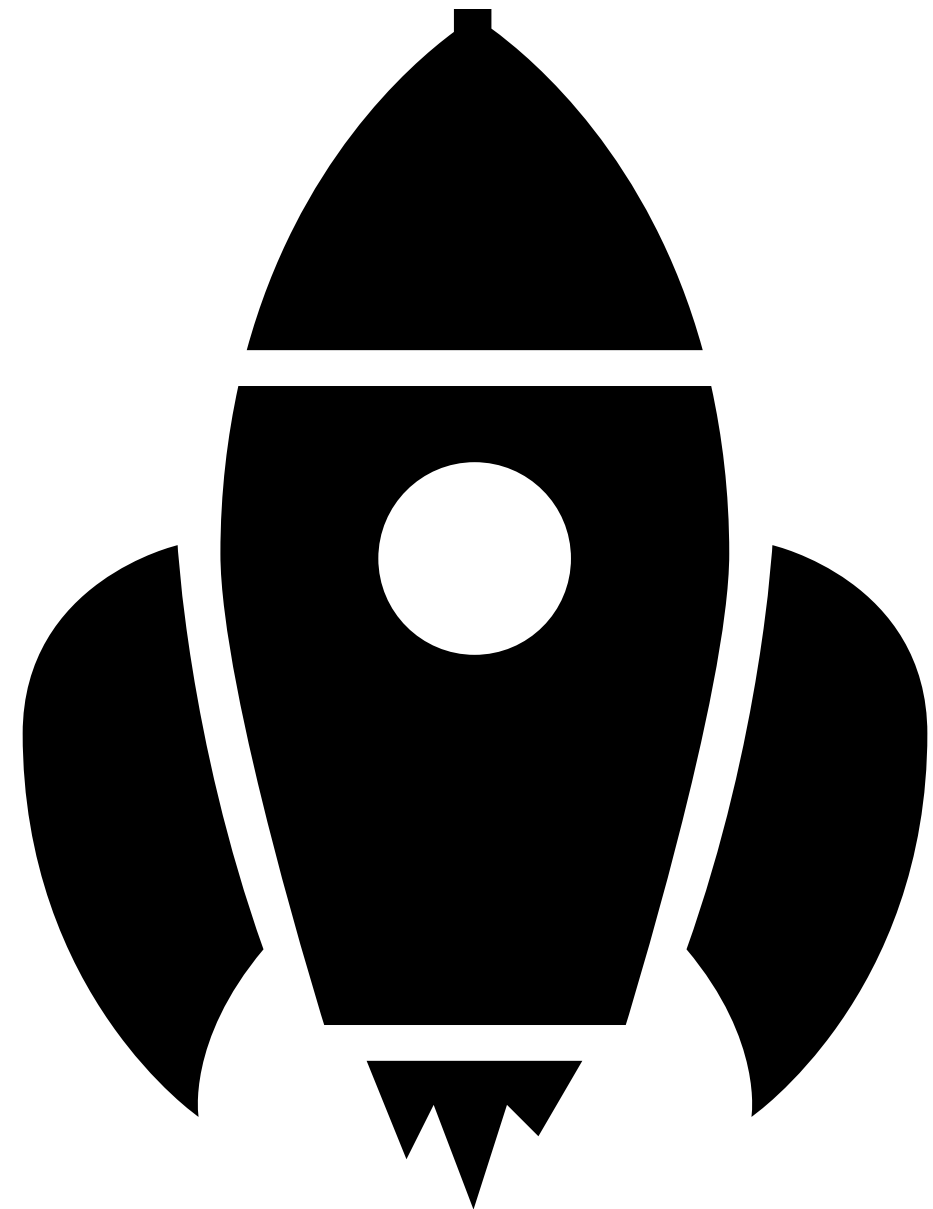
COMPETENCIA



Implementa el uso de interrupciones en sistemas embebidos utilizando Arduino IDE y funciones específicas, para gestionar eventos asincrónicos que requieren atención inmediata.

AGENDA

-  ¿Qué es una interrupción?
-  Interrupciones de hardware y software
-  Interrupciones internas y externas
-  Excepciones
-  Condiciones de disparo
-  Rutinas de servicio de interrupción (ISR)
-  Interrupciones en Arduino



¿QUÉ ES UNA INTERRUPCIÓN?



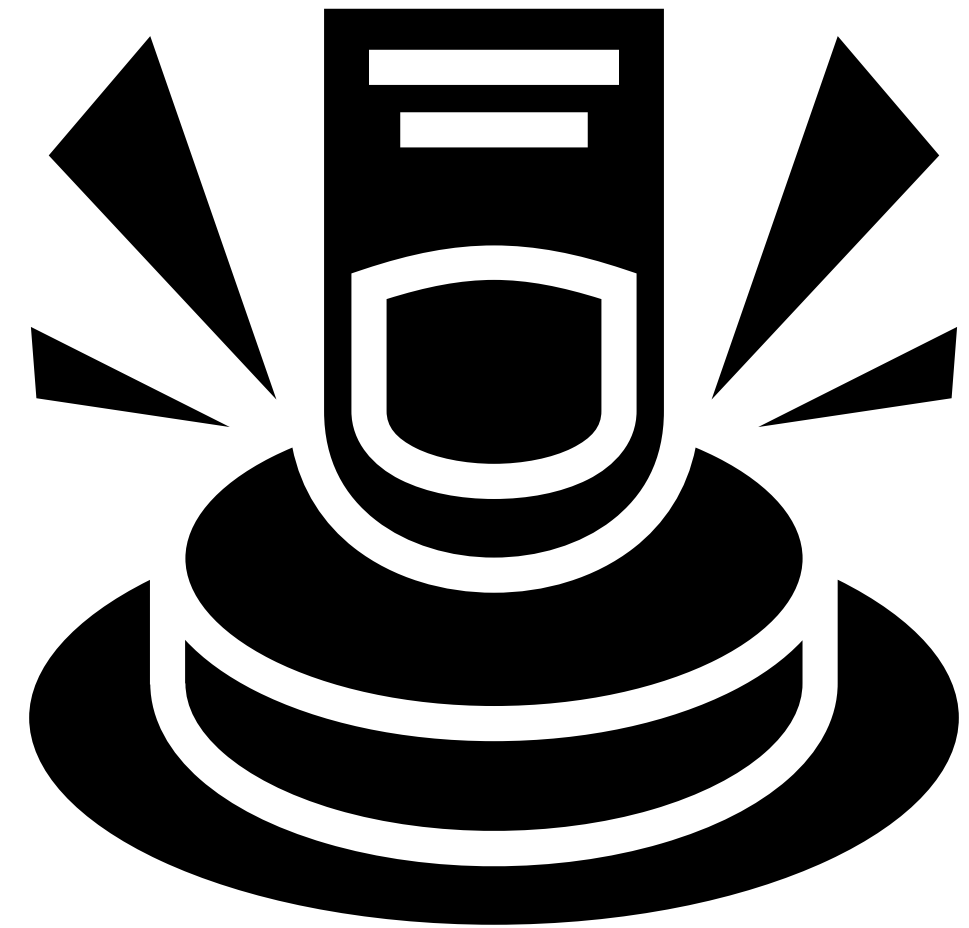
Una interrupción es un evento que pausa temporalmente la ejecución del programa para atender algo urgente.

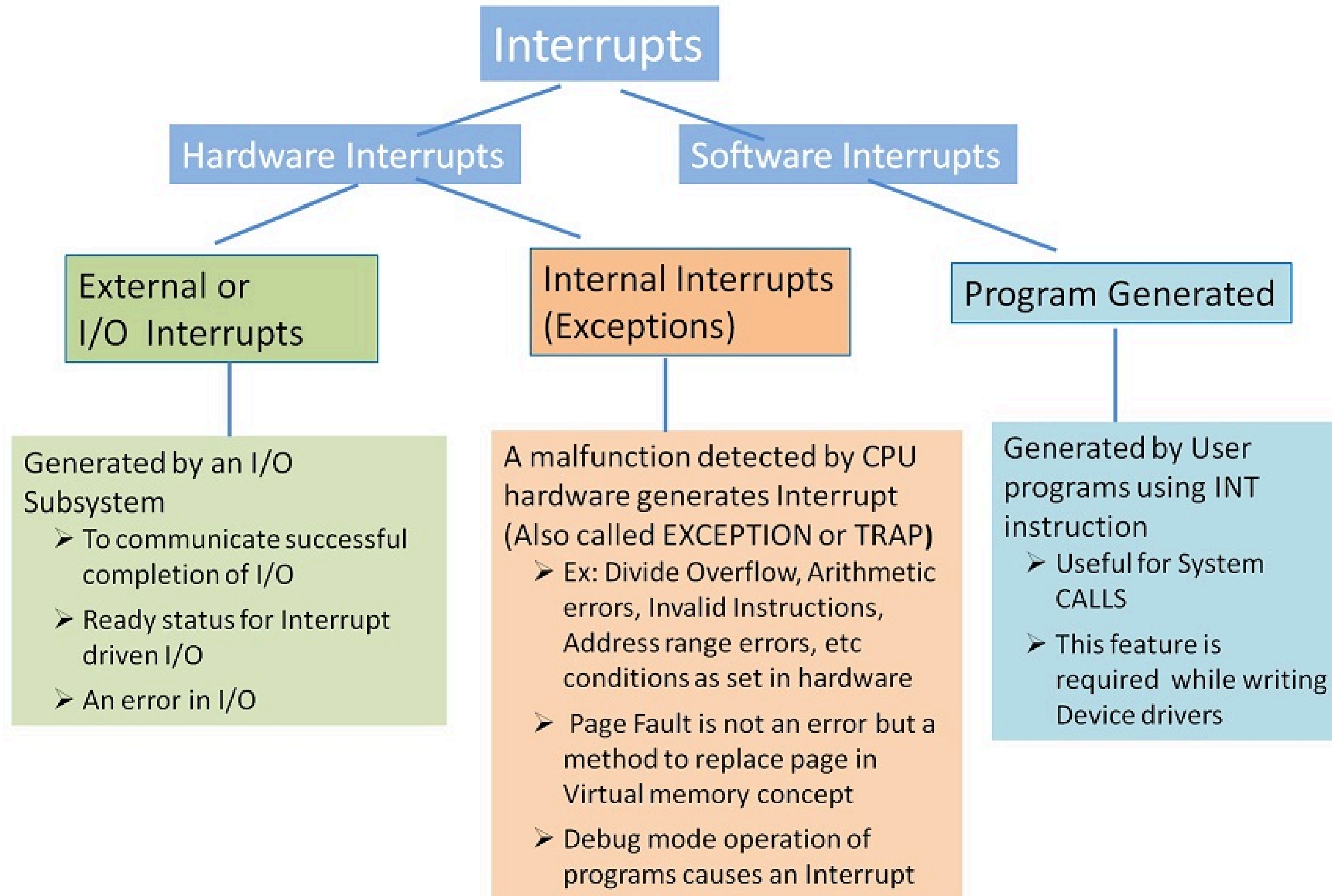
Permite al sistema responder en tiempo real a señales externas o internas.

INTERRUPCIONES DE HARDWARE Y SOFTWARE

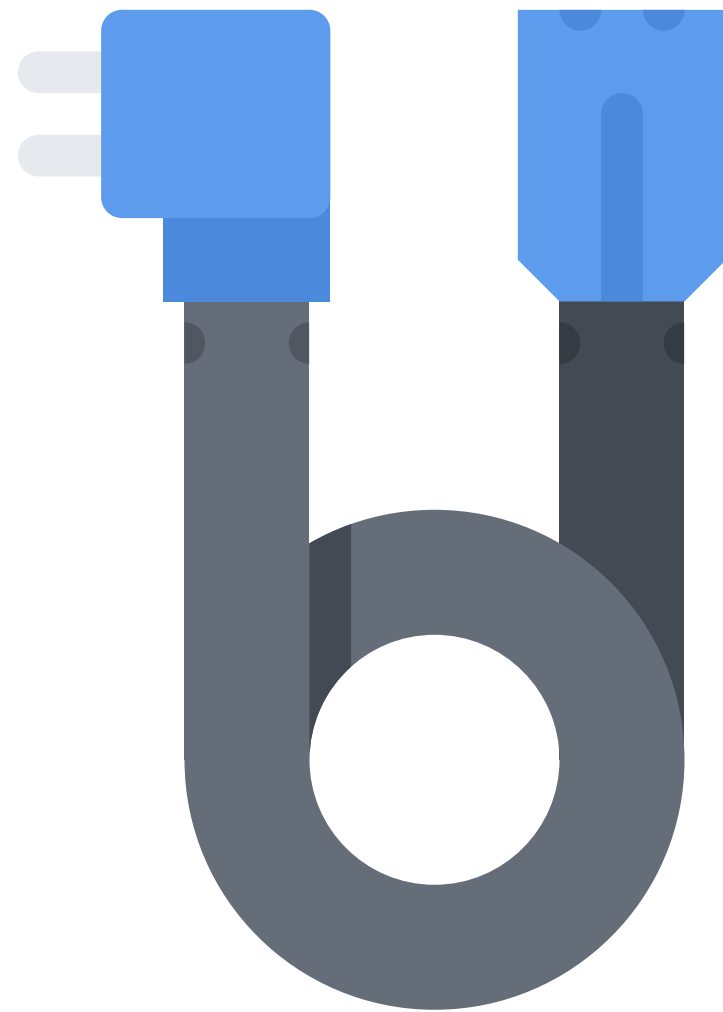
Existen dos tipos principales de interrupciones:

- De hardware: generadas por eventos físicos externos (como un botón o un sensor).
- De software: generadas por el propio código o por el sistema (como un temporizador que se cumple).





INTERRUPCIONES INTERNAS Y EXTERNAS



Las interrupciones pueden clasificarse según su origen físico dentro o fuera del microcontrolador:

- Externas: Proviene de señales fuera del chip (como un botón o sensor en un pin).
- Internas: Generadas dentro del microcontrolador (como temporizadores, errores o eventos del sistema).

EJEMPLOS DE INTERRUPCIONES

Tipo	Ejemplo real en Arduino	¿Qué lo causa?	¿Dónde pasa?
Hardware + Externa	Presionar un botón conectado al pin 2	Botón (físico)	Desde fuera del chip (por un pin)
Hardware + Interna	Arduino recibe datos por USB (Serial/UART)	Módulo UART (físico, interno)	Dentro del chip
Software + Externa	Otro Arduino manda señal al pin 3 usando código	Código externo	Llega por un pin externo
Software + Interna	Temporizador que se cumple (ej. cada 1s)	Código dentro del Arduino	Dentro del chip

EXCEPCIONES



Las excepciones son un tipo de interrupción interna, causada por eventos inesperados o críticos dentro del microcontrolador:

- División por cero
- Desbordamiento (overflow)
- Acceso a memoria inválida
- Fallos en instrucciones del sistema

CONDICIONES DE DISPARO DE UNA INTERRUPTIÓN

Las interrupciones externas se pueden configurar para activarse cuando la señal:

- RISING: pasa de 0 a 1 (subida)
- FALLING: pasa de 1 a 0 (bajada)
- CHANGE: cambia (ya sea subida o bajada)
- LOW: permanece en estado bajo (0V)



EJEMPLOS CON BOTÓN + LED

USANDO INPUT_PULLUP

Condición	¿Cuándo se activa?	¿Qué hace el botón?	¿Qué hace el LED?	¿Resultado visible?
FALLING	Pin pasa de HIGH → LOW	Se presiona el botón	Cambia el estado del LED (ON ↔ OFF)	LED cambia solo al presionar
RISING	Pin pasa de LOW → HIGH	Se suelta el botón	Enciende el LED	LED solo se enciende al soltar
CHANGE	El pin cambia (sube o baja)	Se presiona o se suelta	Cambia el estado del LED (ON ↔ OFF)	LED cambia cada vez que tocás el botón
LOW	Mientras el pin esté en LOW	Botón está presionado	Enciende el LED todo el tiempo que se presione	LED se mantiene encendido mientras se presiona

RUTINAS DE SERVICIO DE INTERRUPCIÓN (ISR)

Una ISR es una función que Arduino ejecuta automáticamente cuando ocurre una interrupción.

- Debe ser lo más corta posible.
- No puede tener `delay()`, `Serial.print()` u otras funciones que dependan del tiempo.
- Se escribe sin parámetros ni retorno: `void miISR() { /* instrucciones */ }`



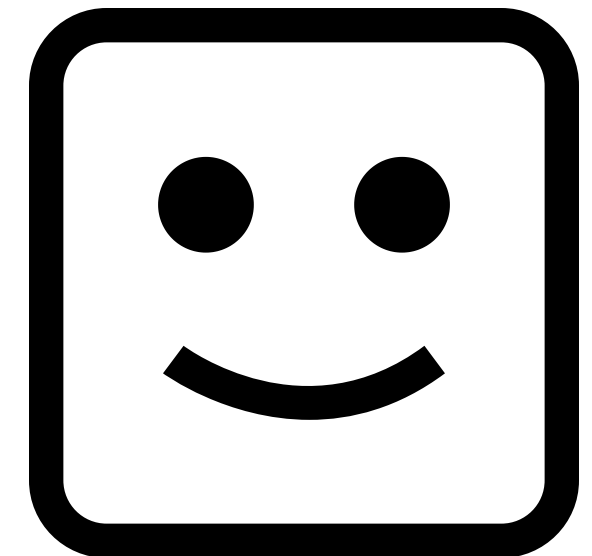
```
attachInterrupt(digitalPinToInterrupt(2), miISR, FALLING);
```

PINES DE ARDUINO

Modelo	INT0	INT1	INT2	INT3	INT4	INT5
UNO	Pin 2	Pin 3				
Nano	Pin 2	Pin 3				
Mini Pro	Pin 2	Pin 3				
Mega	Pin 2	Pin 3	Pin 21	Pin 20	Pin 19	Pin 18
Leonardo	Pin 3	Pin 2	Pin 0	Pin 1	Pin 7	
Due	En todos los pines					

CONCEPTOS CLAVE APRENDIDOS

-  Comprendió qué es una interrupción y cómo interfiere en el flujo normal del programa.
-  Identificó los tipos de interrupciones: de hardware, software, internas y externas.
-  Reconoció cómo funciona una rutina de servicio de interrupción (ISR).
-  Diferenció las condiciones de disparo: RISING, FALLING, CHANGE y LOW.
-  Usó `attachInterrupt()` para vincular eventos con funciones en tiempo real.
-  Conoció las limitaciones dentro de una ISR (no usar `delay`, ni `Serial`, etc).
-  Aplicó un ejemplo funcional con botón e ISR para controlar un LED.



NOMBRE DEL VALOR Y/O ACTITUD

Autonomía:

Permite diseñar soluciones que reaccionan automáticamente a eventos, sin depender de la intervención continua del usuario. Refuerza la idea de que un sistema puede actuar por sí mismo ante situaciones críticas o importantes.

Precisión:

Desarrolla la capacidad de tomar decisiones técnicas correctas sobre cuándo y cómo debe reaccionar un sistema. Fomenta el pensamiento lógico y el control riguroso de condiciones específicas.



**¡GRACIAS POR
SU ATENCIÓN!
¿DUDAS?**



**RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES
CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA SEMANA.**